

TREE 2

PROBLEM 3

QUESTION:

DS

Session

Tree 2

Question Information

Level 1

Challenge 73

Question Description:

There are n cities and m flight connections between them. Your task is to add new flights so that it will be possible to travel from any city to any other city. What is the minimum number of new flights required?

Input

The first input line has two integers n and m : the number of cities and flights. The cities are numbered $1, 2, \dots, n$.

After this, there are m lines describing the flights. Each line has two integers a and b : there is a flight from city a to city b . All flights are one-way flights.

Output

First print an integer k : the required number of new flights. After this, print k lines describing the new flights. You can print any valid solution.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Logical Test Cases

Test Case 1

INPUT (STDIN)

4 5
1 2
2 3
3 1
1 4
3 4

EXPECTED OUTPUT

1
4 1

Test Case 2

INPUT (STDIN)

4 5
1 3
2 1
1 2
2 4
1 4

EXPECTED OUTPUT

2
4 3
3 1

Mandatory Test Cases

Test Case 1

KEYWORD

while(m--)

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

1

Test Case 2

TOKEN COUNT

1080

Test Case 3

NLOC

130

SOURCE CODE:

```
#include <stdio.h>

int root_parent[100005];

int find_root(int node) {
    if (root_parent[node] == node) return node;
    return root_parent[node] = find_root(root_parent[node]);
}

void join_roots(int a, int b) {
    int ra = find_root(a);
    int rb = find_root(b);
    if (ra != rb) root_parent[rb] = ra;
}

int main() {
    int total_cities, flights;
    if (scanf("%d %d", &total_cities, &flights) != 2) return 0;

    for (int i = 1; i <= total_cities; i++) {
        root_parent[i] = i;
    }

    while (flights--) {
        int origin, dest;
        scanf("%d %d", &origin, &dest);
        join_roots(origin, dest);
    }

    int leaders[100005];
    int comp_count = 0;
    for (int i = 1; i <= total_cities; i++) {
        if (find_root(i) == i) {
            leaders[comp_count++] = i;
        }
    }

    int required_flights = comp_count - 1;
    printf("%d\n", required_flights);
    for (int i = 0; i < required_flights; i++) {
        printf("%d %d\n", leaders[i], leaders[i + 1]);
    }
    return 0;
}
```

OUTPUT:

```
4 5
1 4
2 4
3 1
3 4
4 1
2
4 2
4 3
```

```
4 5
1 3
2 1
1 2
2 4
1 4
2
4 3
3 1
```